

BLOCKCHAIN EXPERIMENT - 11

AIM: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

THEORY:

Introduction to Hyperledger Fabric

Hyperledger Fabric is an open-source, enterprise-grade permissioned blockchain framework hosted under the Linux Foundation's Hyperledger umbrella. Unlike public blockchains such as Ethereum or Bitcoin where anyone can join the network anonymously, Hyperledger Fabric operates on a membership-based model where every participant must be authenticated and authorized before joining. This makes it particularly suitable for business and enterprise use cases where privacy, accountability, and controlled access are critical requirements. Fabric supports modular architecture, meaning components like consensus mechanisms, membership services, and data storage can be swapped or customized based on the needs of the network.

Key Components of a Hyperledger Fabric Network

A Hyperledger Fabric network consists of several interconnected components that work together to process and record transactions.

Peers are the fundamental nodes of the network. Each organization in the network operates one or more peer nodes that maintain a copy of the ledger and execute chaincode. Peers can serve two roles — an endorsing peer which simulates and endorses transactions, and a committing peer which validates and commits blocks to the ledger.

The Orderer (also called the Ordering Service) is responsible for establishing consensus on the order of transactions and packaging them into blocks. It does not execute chaincode or maintain the full ledger state, but acts as the backbone for consistent block delivery across all peers.

The Certificate Authority (CA) manages digital identities within the network. Every participant, whether a peer, orderer, or user, must possess a valid certificate issued by the CA to interact with the network. Fabric uses its own CA implementation called Fabric CA.

Channels are a privacy mechanism unique to Hyperledger Fabric. A channel is essentially a private subnet of communication between specific organizations. Transactions on one channel are completely invisible to organizations not participating in that channel, enabling data isolation within a shared network infrastructure.

The Membership Service Provider (MSP) defines the rules for validating identities and determining which entities are members of the network or a specific channel. It maps certificates to organizational roles and permissions.

Chaincode (Smart Contracts)

Chaincode is the term used in Hyperledger Fabric for smart contracts. It is application-level code that defines the business logic of the network — specifying how assets are created, updated, transferred, and queried. Chaincode runs inside Docker containers on peer nodes and interacts with the ledger through a well-defined API. It can be written in multiple languages including Go, JavaScript (Node.js), and Java. Before chaincode can be used, it must go through a lifecycle process that includes packaging, installing on peers, approving by the required organizations, and finally committing to the channel.

The Ledger

The Hyperledger Fabric ledger consists of two components. The first is the blockchain, which is an immutable, append-only log of all transactions since the genesis block. The second is the world state, which is a database (LevelDB or CouchDB) that holds the current values of all assets. When a transaction is committed, the world state is updated to reflect the latest asset values, while the full transaction history remains preserved in the blockchain log.

Transaction Flow in Hyperledger Fabric

The transaction flow in Hyperledger Fabric follows a structured execute-order-validate pattern which is distinct from other blockchain platforms.

First, a client application submits a transaction proposal to one or more endorsing peers. The endorsing peers simulate the transaction by executing the chaincode against the current ledger state and return a signed endorsement response without actually updating the ledger.

Second, once the client has collected sufficient endorsements as required by the endorsement policy, it submits the endorsed transaction to the ordering service. The orderer does not validate the business logic but simply orders the transactions and packages them into a block.

Third, the orderer broadcasts the block to all peers on the channel. Each peer independently validates every transaction in the block, checking that endorsement policies are satisfied and that the read-write sets are consistent with the current ledger state. Valid transactions are committed to the ledger and the world state is updated accordingly. Invalid transactions are marked but still recorded.

Fabric Test Network

The Hyperledger Fabric test network is a pre-configured sample network provided in the fabric-samples repository. It consists of two organizations (Org1 and Org2), each with one peer node, and a single orderer node. It uses Raft consensus for ordering and is designed for development and educational purposes. The `network.sh` script automates the process of starting the network, creating channels, deploying chaincode, and shutting down the network.

Asset Transfer Chaincode

The asset-transfer-basic chaincode used in this experiment demonstrates fundamental ledger operations. It defines an asset with properties such as ID, color, size, owner, and appraised value. The chaincode exposes functions including InitLedger to populate the ledger with sample assets, CreateAsset to add new assets, ReadAsset to fetch a specific asset by ID, GetAllAssets to retrieve all assets, TransferAsset to update the owner of an asset, and UpdateAsset to modify asset properties.

IMPLEMENTATION:

Step 1: Update the system

Go to <https://shell.cloud.google.com> and run the following commands:

```
sudo apt update
```

```
sudo apt upgrade -y
```

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
To set your Cloud Platform project in this session use `gcloud config set project [PROJECT_ID]`.
You can view your projects by running `gcloud projects list`.
g2022_prajjwal_pandey@cloudshell:~$ sudo apt update
sudo apt upgrade -y
Get:1 https://cli.github.com/packages stable InRelease [3,917 B]
Hit:2 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease
Get:3 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:4 https://packages.cloud.google.com/apt gcsfuse-noble InRelease [1,227 B]
Get:5 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [59.7 kB]
Get:6 https://packages.cloud.google.com/apt cloud-sdk InRelease [1,620 B]
Get:7 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease [180 kB]
Hit:8 http://archive.ubuntu.com/ubuntu noble InRelease
Get:9 https://packages.cloud.google.com/apt gcsfuse-noble/main amd64 Packages [66.1 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:11 https://apt.postgresql.org/pub/repos/apt noble-pgdg/main amd64 Packages [675 kB]
Get:12 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease [3,005 B]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:14 https://packages.cloud.google.com/apt cloud-sdk/main all Packages [1,982 kB]
Get:15 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0/multiverse amd64 Packages [68.7 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2,377 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [3,505 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [3,692 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2,155 kB]
Get:21 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,989 kB]
Get:22 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,508 kB]
Get:23 https://packages.cloud.google.com/apt cloud-sdk/main amd64 Packages [4,640 kB]
Fetched 23.3 MB in 10s (2,440 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
61 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Step 2: Install Required Dependencies

```
g2022_prajwal_pandey@cloudshell:~$ sudo apt install -y jq curl git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
curl is already the newest version (8.5.0-2ubuntu10.8).
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
g2022_prajwal_pandey@cloudshell:~$
```

Then verify everything is present:

```
node -v
npm -v
git --version
docker --version
docker ps
```

```
g2022_prajwal_pandey@cloudshell:~$ node -v
v20.11.1
npm -v
11.12.1
git --version
git version 2.43.0
docker --version
Docker version 29.4.0, build 9d7ad9f
g2022_prajwal_pandey@cloudshell:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
g2022_prajwal_pandey@cloudshell:~\$						

Step 3: Fix Docker Compose

```
DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
mkdir -p $DOCKER_CONFIG/cli-plugins
curl -SL
https://github.com/docker/compose/releases/download/v2.24.5/docker-compose-linux-x86_64 \
-o $DOCKER_CONFIG/cli-plugins/docker-compose
chmod +x $DOCKER_CONFIG/cli-plugins/docker-compose
docker compose version
sudo mv /usr/bin/docker-compose /usr/bin/docker-compose-old 2>/dev/null ||
true
sudo tee /usr/local/bin/docker-compose > /dev/null <<'EOF'
#!/bin/bash
exec docker compose "$@"
EOF
sudo chmod +x /usr/local/bin/docker-compose
```

docker compose version

docker-compose version

```
g2022_prajjwal_pandey@cloudshell:~$ DOCKER_CONFIG=${DOCKER_CONFIG:-$HOME/.docker}
mkdir -p $DOCKER_CONFIG/cli-plugins
curl -SL https://github.com/docker/compose/releases/download/v2.24.5/docker-compose-linux-x86_64 \
-o $DOCKER_CONFIG/cli-plugins/docker-compose
chmod +x $DOCKER_CONFIG/cli-plugins/docker-compose
docker compose version
% Total      % Received % Xferd  Average Speed   Time    Time     Time  Current
   Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--    0
100 58.5M 100 58.5M    0     0 37.1M      0  0:00:01  0:00:01 --:--:-- 52.7M
Docker Compose version v2.24.5
g2022_prajjwal_pandey@cloudshell:~$ sudo mv /usr/bin/docker-compose /usr/bin/docker-compose-old 2>/dev/null || true
sudo tee /usr/local/bin/docker-compose > /dev/null <<'EOF'
#!/bin/bash
exec docker compose "$@"
EOF
sudo chmod +x /usr/local/bin/docker-compose
docker-compose version
Docker Compose version v2.24.5
g2022_prajjwal_pandey@cloudshell:~$
```

Step 4: Clone Fabric Samples

cd \$HOME

git clone https://github.com/hyperledger/fabric-samples

cd fabric-samples

```
g2022_prajjwal_pandey@cloudshell:~$ cd $HOME
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (221/221), done.
remote: Compressing objects: 100% (126/126), done.
remote: Total 15618 (delta 149), reused 95 (delta 95), pack-reused 15397 (from 3)
Receiving objects: 100% (15618/15618), 24.09 MiB | 14.31 MiB/s, done.
Resolving deltas: 100% (8801/8801), done.
g2022_prajjwal_pandey@cloudshell:~/fabric-samples$
```

Step 5: Install Fabric Binaries and Docker Images

curl -sSLO

<https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install>

fabric.sh

chmod +x install-fabric.sh

./install-fabric.sh docker samples binary

```

g2022_prajwal_pandey@cloudshell:~/fabric-samples$ curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

==> Downloading version 2.5.15 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
==> Will unpack to: /home/g2022_prajwal_pandey/fabric-samples
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 120M 100 120M    0     0 32.5M      0  0:00:03 0:00:03 --:--:-- 37.3M
==> Done.
==> Downloading version 1.5.17 platform specific fabric-ca-client binary
==> Downloading: https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
==> Will unpack to: /home/g2022_prajwal_pandey/fabric-samples
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 31.6M 100 31.6M    0     0 25.3M      0  0:00:01 0:00:01 --:--:-- 64.4M
==> Done.

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
==> Pulling fabric Images
==> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
b1c8a2e842ca: Pull complete
eb2bee6aa5bd: Pull complete

```

```

b9649d3f054a: Pull complete
6f4ebca3e823: Pull complete
18afb8c8d2a4: Pull complete
0b966e606d9a: Pull complete
c36bacd85367: Download complete
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
==> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9    250MB    80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17       453a0a727e82    381MB    123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92    1GB      255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47    178MB    49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75    230MB    66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9    250MB    80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9    250MB    80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9    250MB    80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82    381MB    123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82    381MB    123MB
hyperledger/fabric-ca:latest                453a0a727e82    381MB    123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92    1GB      255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92    1GB      255MB
hyperledger/fabric-ccenv:latest             86dded7eda92    1GB      255MB
hyperledger/fabric-orderer:2.5              9972c209be47    178MB    49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47    178MB    49.5MB
hyperledger/fabric-orderer:latest           9972c209be47    178MB    49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75    230MB    66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75    230MB    66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75    230MB    66.2MB

```

Step 6: Verify Docker images

docker images | grep fabric

```

g2022_prajjwal_pandey@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17         453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15      86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15    9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15             654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                    453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                 453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                  453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                 86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15              86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB        255MB
hyperledger/fabric-orderer:2.5               9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15            9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest             9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                  3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15               3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB
g2022_prajjwal_pandey@cloudshell:~/fabric-samples$

```

Step 7: Set Environment Variables

```

export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version

```

```

g2022_prajjwal_pandey@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: gol.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger
g2022_prajjwal_pandey@cloudshell:~/fabric-samples$

```

Step 8: Start the Network and Create Channel

```

cd $HOME/fabric-samples/test-network
./network.sh up createChannel

```

```

g2022_prajwal_pandey@cloudshell:~/fabric-samples$ cd $HOME/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb with crypto from 'cryptogen'
Bringing up network
LOCAL VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/g2022_prajwal_pandey/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] Running 3/7
! Network fabric_test Created 0.8s
! Volume "compose_peer0.org1.example.com" Created 0.8s
! Volume "compose_peer0.org2.example.com" Created 0.8s
! Volume "compose_orderer.example.com" Created 0.8s
✓ Container peer0.org2.example.com Started 0.8s
✓ Container peer0.org1.example.com Started 0.6s
✓ Container orderer.example.com Started 0.7s
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
1443abe62100   hyperledger/fabric-peer:latest      "peer node start"       1 second ago   Up Less than a second   0.0.0.0:7051->7051/tcp, 0.0.0.0:9444->9444/tcp
259aa87d5074   hyperledger/fabric-peer:latest      "peer node start"       1 second ago   Up Less than a second   0.0.0.0:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp
45/tcp        peer0.org2.example.com

```

```

/home/g2022_prajwal_pandey/fabric-samples/test-network/../bin/configtxgen
+ '[' 0 -eq 1 ']'
+ configtxgen -profile ChannelUsingRaft -outputBlock ./channel-artifacts/mychannel.block -channelID mychannel
2026-04-10 04:21:22.361 UTC 0001 INFO [common.tools.configtxgen] main -> Loading configuration
2026-04-10 04:21:22.371 UTC 0002 INFO [common.tools.configtxgen.localconfig] completeInitialization -> orderer type: etcdraft
2026-04-10 04:21:22.371 UTC 0003 INFO [common.tools.configtxgen.localconfig] completeInitialization -> Orderer.EtcdRaft.Options unset, setting to tick_interval:
"500ms" election_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-10 04:21:22.371 UTC 0004 INFO [common.tools.configtxgen.localconfig] Load -> Loaded configuration: /home/g2022_prajwal_pandey/fabric-samples/test-netwo
rk/configtx/configtx.yaml
2026-04-10 04:21:22.376 UTC 0005 INFO [common.tools.configtxgen] doOutputBlock -> Generating genesis block
2026-04-10 04:21:22.376 UTC 0006 INFO [common.tools.configtxgen] doOutputBlock -> Creating application channel genesis block
2026-04-10 04:21:22.377 UTC 0007 INFO [common.tools.configtxgen] doOutputBlock -> Writing genesis block
+ res=0
Creating channel mychannel
Adding orderers
+ . scripts/orderer.sh mychannel
+ '[' 0 -eq 1 ']'
+ res=0
Status: 201
{
  "name": "mychannel",
  "url": "/participation/v1/channels/mychannel",
  "consensusRelation": "consenter",
  "status": "active",
  "height": 1
}
Channel 'mychannel' created
Joining org1 peer to the channel...
Using organization 1
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-10 04:21:28.558 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-10 04:21:28.617 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted proposal to join channel
Joining org2 peer to the channel...
Using organization 2

```

```
-- jq '.data.data[0].payload.data.config' /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/config_block.json
++ res=0
Generating anchor peer update transaction for Org2 on channel mychannel
-- jq '.channel.group.groups.Application.groups.OrgMSP.values ++ [{"AnchorPeers":{"mod_policy": "Admins","value":[{"anchor_peers":["{\"host\": \"peer0.org2.example.com\", \"port\": 9051}],\"version\":\"0\"}"]}' /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/OrgMSPconfig.json
++ res=0
++ configtxlator proto_encode --input /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/OrgMSPconfig.json --type common.Config --outputpb /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/original_config.pb
++ configtxlator proto_encode --input /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/OrgMSPmodified_config.json --type common.Config --outputpb /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/modified_config.pb
++ configtxlator compute_update --channel id mychannel --original /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/original_config.pb --updated /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/modified_config.pb --output /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/config_update.pb
++ configtxlator proto_decode --input /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/config_update.pb --type common.ConfigUpdate --outputpb /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/config_update.json
-- jq .
+++ cat /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/config_update.json
++ echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel","type":2},"data":{"config_update":{"channel_id":"mychannel","isolated_data":{"read_set":{"groups":{"Application":{"groups":{"OrgMSP":{"groups":{"mod_policy":{"policy":"null","version":"0"},"Endorsement":{"mod_policy":{"policy":"null","version":"0"},"Readers":{"mod_policy":{"policy":"null","version":"0"},"values":{"MSB":{"mod_policy":{"value":"null","version":"0"},"version":"0"},"mod_policy":{"value":"null","version":"0"},"write set":{"groups":{"Application":{"groups":{"OrgMSP":{"groups":{"mod_policy":{"policy":"Admins","policies":{"Admins":{"mod_policy":{"policy":"null","version":"0"},"Endorsement":{"mod_policy":{"policy":"null","version":"0"},"Readers":{"mod_policy":{"policy":"null","version":"0"},"values":{"AnchorPeers":{"mod_policy":{"Admins","value":{"anchor_peers":[{"host":"peer0.org2.example.com","port":9051}]},{"port":9051}]},"version":"0"},"MSB":{"mod_policy":{"value":"null","version":"0"},"version":"1"},"mod_policy":{"value":"null","version":"0"},"values":{"MSB":{"mod_policy":{"value":"null","version":"0"},"mod_policy":{"value":"null","version":"0"},"policies":{"values":{"version":"0"}}}}}}]]}}}}}}}}}}}}'}'
++ configtxlator proto_encode --input /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Envelope --output /home/g2022_prajwal_pandey/fabric-samples/test-network/channel-artifacts/OrgMSPanchors.tx
2026-04-10 04:21:32.510 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-10 04:21:32.530 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
g2022_prajwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 9: Deploy Chaincode

```
./network.sh deployCC \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccl javascript
```

```
g2022_prajwal_pandey@cloudshell:~/fabric-samples/test-network$ ./network.sh deployCC \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-cc1 javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
```

```
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/g2022_prajwal_pandey/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --name basic --tlsRootCertFiles /home/g2022_prajwal_pandey/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAddresses localhost:9051 --tlsRootCertFiles /home/g2022_prajwal_pandey/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --version 1.0 --sequence 1
+ res=0
2026-04-10 04:30:01.957 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [25c6828e7e229e5993e8ced76a95f4db766b07bb241b4e8623c87be... (VALID) at localhost:9051
2026-04-10 04:30:01.957 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [25c6828e7e229e5993e8ced76a95f4db766b07bb241b4e8623c87be... (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
g2022_prajwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 10: Set Peer Environment Variables

```
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

```
g2022_prajwal_pandey@cloudshell:~/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
g2022_prajwal_pandey@cloudshell:~/fabric-samples/test-network$
```

Step 11: Initialize Ledger

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem \
-C mychannel \
```

```

-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
-c '{"function": "InitLedger", "Args": []}'

```

```

g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp/peers/peer0.org1.example.com
export CORE_PEER_ADDRESS=localhost:7051
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tls/ca.crt \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function": "InitLedger", "Args": []}'
2026-04-10 04:31:11.891 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$

```

Step 12: Query All Assets

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```

g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad",
"Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"
yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asse
t"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$

```

Step 13: Read a Specific Asset

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```

```

g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$

```

Step 14: Create a New Asset

```

peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \

```

```

--cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
-c '{"function": "CreateAsset", "Args": ["asset7", "blue", "10", "Prajwal", "500"]}'

```

```

g2022_prajwal_pandey@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.p
em \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function": "CreateAsset", "Args": ["asset7", "blue", "10", "Prajwal", "500"]}'
2026-04-10 04:32:26.761 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"asset7","Col
or":"blue","Size":10,"Owner":"Prajwal","AppraisedValue":500}
g2022_prajwal_pandey@cloudshell:~/fabric-samples/test-network$

```

Step 15: Transfer Asset

```

peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/
peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/
peer0.org2.example.com/tls/ca.crt \
-c '{"function": "TransferAsset", "Args": ["asset7", "NewOwnerName"]}'

```

```

g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function": "TransferAsset", "Args": [{"asset7", "NewOwnerName"}]}'
2026-04-10 04:32:53.864 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Prajjwal"
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$

```

Step 16: Stop the Network

cd \$HOME/fabric-samples/test-network

./network.sh down

```

g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$ cd $HOME/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
project name can't be empty. Use `--project-name` to set a valid name
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
1443abe62100
259aa87d5074
5f813b7c36d4
2064481a2e64
d543cc06ae09
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbc664c87b24c035bee15cbf1398b52d341cb:latest
Deleted: sha256:61e84acc6a2ea295147c3ef35b6f3c35696344c81322bb21a8fd31808d74cbfd2
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbc664c87b24c035bee15cbf35f604aa885d60dc:latest
Deleted: sha256:9572fbfb8ae5a8987340d8ce12ad3f5ee8600acb74db3413496880b7bc0fc6d4
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
481282afbc43: Pull complete
aecba0016ef2: Download complete
Digest: sha256:1487d0af5f52b4ba31c7e465126ee2123fe3f2305d638e7827681e7cf6c83d5e
Status: Downloaded newer image for busybox:latest
g2022_prajjwal_pandey@cloudshell:~/fabric-samples/test-network$

```

CONCLUSION

This experiment successfully demonstrated the setup and operation of a Hyperledger Fabric network using the fabric-samples test network. The network was configured with two organizations, each running a peer node, along with a Raft-based ordering service responsible for block production and transaction sequencing. Docker Compose was used to orchestrate all the network components as containerized services.

The asset-transfer-basic chaincode written in JavaScript was deployed following the complete Fabric chaincode lifecycle — packaging, installation, organizational approval, and channel commitment. Once the ledger was initialized using the InitLedger function, a set of pre-defined assets became available on the channel. Various chaincode operations were then performed including querying all assets using GetAllAssets, retrieving a specific asset using ReadAsset,

creating a new asset with custom attributes using `CreateAsset`, and transferring asset ownership to a new participant using `TransferAsset`.

This experiment provided practical exposure to the core concepts of enterprise blockchain development including permissioned network setup, certificate-based identity management, chaincode deployment and invocation, and the execute-order-validate transaction flow that distinguishes Hyperledger Fabric from other blockchain platforms. The experiment also highlighted how Fabric's modular and privacy-oriented architecture makes it well suited for real-world business applications involving multiple organizations with varying trust boundaries.